Accelerate AI Inference

2026 | Updates are here.
Post your questions here.
Read the documentation here.

OpenVINO™

Overview Cheat Sheet

Bring AI everywhere with OpenVINO™: enabling developers to quickly optimize, deploy, and scale AI applications across hardware device types with cutting-edge compression features and advanced performance capabilities.

What is OpenVINO™?

OpenVINO is an open-source toolkit for optimizing and deploying deep learning models.

Deploy AI across devices (from PC to cloud) with automatic acceleration!

Documentation | Get Started | Blog | Examples

Use OpenVINO™ with...

PyTorch | TensorFlow | Hugging Face | ONNX | and more

Build, Optimize, Deploy

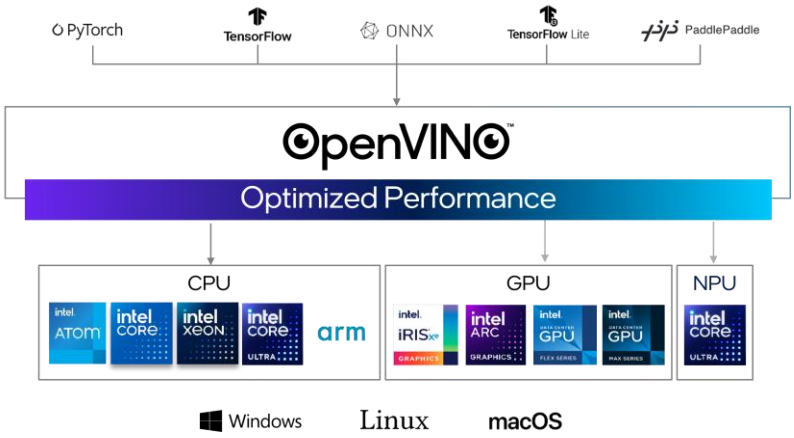
OpenVINO™ accelerates inference and simplifies deployment across hardware, with a "**build once, deploy everywhere**" philosophy. To accomplish this, OpenVINO supports + integrates with frameworks (like PyTorch) and offers advanced compression capabilities.

Build your model in the training framework or grab a pre-trained model from Hugging Face

Optimize your model for faster responses & smaller memory

Deploy the same model across hardware, with automatic performance enhancements

Leverage the hardware's AI acceleration by default



OpenVINO™ Installation

Linux Install | Windows Install | macOS Install

PyPI example for Linux, macOS & Windows:

```
#set up python venv
python -m pip install openvino
```

The install table also has: PyPI, APT, YUM, & npm

Interactive Notebook Examples

Test out 150+ interactive Jupyter notebooks with cutting-edge open-source models.

Includes model compression, pipeline details, interactive GUIs, and more.

Try out top models for a range of use cases, including:

LLM | YOLO-v11 | MCP Agent | VLM | Paddle OCR | Whisper

Setup: Windows | Ubuntu | macOS | RedHat | CentOS | AzureML | Docker | SageMaker

Model Compression with NNCF

NNCF is OpenVINO™'s deep learning model compression tool, offering cutting-edge AI compression capabilities, including:

1. **Quantization**: reducing the bit-size of the weights, while preserving accuracy
2. **Weight Compression:** easy post-training optimization for LLMs+
3. **Pruning for Sparsity**: drop connections in the model that don't add value
4. **Model Distillation**: a larger 'teacher' model trains a smaller 'student' model

Compression results in smaller and faster models that can be deployed across devices.

Easy install: `pip install nncf`

Documentation | GitHub* | NNCF Notebooks | NNCF + Hugging Face*

PyTorch + OpenVINO Options

PyTorch models can be directly converted within OpenVINO™:

```
import openvino as ov
import torch
model = torch.load("model.pt") # Convert model loaded from PyTorch file
model.eval()
ov_model = ov.convert_model(model)
core = ov.Core()
compiled_model = core.compile_model(ov_model) # Compile model from memory
```

Or you can use the OpenVINO backend for torch.compile:

```
import openvino.torch
import torch
# Compile PyTorch model
opts = {"device" : "CPU", "config" : {"PERFORMANCE_HINT" : "LATENCY"}}
compiled_model = torch.compile(model, backend="openvino", options=opts)
```

Direct Conversion | PyTorch Backend | Examples | Blog

Performance Features

OpenVINO™ can do automatic performance enhancements at runtime customized to your hardware (preserving model accuracy), including:

Asynchronous execution, batch processing, tensor fusion, load balancing, dynamic inference parallelism, automatic BF16 conversion (for AMX), and more.

Results in a smaller memory footprint of framework + model, for deploying on small devices.

There are also optional security features: the ability to compute on an encrypted model.

Additional advanced performance features:

- Automatic Device Selection (AUTO) selects the best available device
- Heterogeneous Execution (HETERO) efficiently splits inference between cores
- Automatic Batching ad-hoc groups inference requests for max memory/core utilization
- Performance Hints auto-adjusts runtime parameters to prioritize latency or throughput
- Dynamic Shapes reshapes models to accept arbitrarily-sized inputs, for data flexibility
- Benchmark Tool characterizes model performance in various hardware and pipelines

Supported Hardware

OpenVINO™ supports CPU, GPU, and NPU. (Specifications)

CPU: intel Atom, intel Core, intel Xeon, intel Core Ultra, arm

GPU: intel Iris Xe Graphics, intel Arc Graphics, intel Data Center GPU Flex Series, intel Data Center GPU Max Series

NPU: intel Core Ultra

The plugin architecture of OpenVINO enables development and plug-independent inference solutions dedicated to different devices. Learn more about the Plugin, OpenVINO Plugin Library, and how to build a new plugin with CMake.

Additional community-supported plugins for Nvidia, Java, and Rust can be found here.

OpenVINO can Accelerate as a Backend

If you want to stay in another framework API, OpenVINO™ provides accelerating backends:

PyTorch	Compile PyTorch models with torch.compile using the OpenVINO backend.
ExecuTorch	Export and run AI models with ExecuTorch using the OpenVINO backend on Intel hardware.
ONNX Runtime	Run ONNX models through the OpenVINO Execution Provider for ONNX Runtime.
WinML	Use OpenVINO through Windows ML and ONNX Runtime execution providers for local Windows AI inference.
Hugging Face	Use Optimum Intel to export or load OpenVINO models through familiar Hugging Face APIs.
Nvidia Triton	Serve models in Triton with the OpenVINO backend configuration.
LangChain	Build LangChain pipelines with Hugging Face pipelines using backend="openvino".

Hugging Face Integration

Hugging Face + Optimum Intel integrates OpenVINO with Transformers models and pipelines. Use pre-optimized OpenVINO models from Hugging Face or export models with Optimum Intel, then apply OpenVINO Runtime and compression features through familiar Hugging Face APIs.

For generative AI workflows, see the OpenVINO GenAI Quick-start Guide.

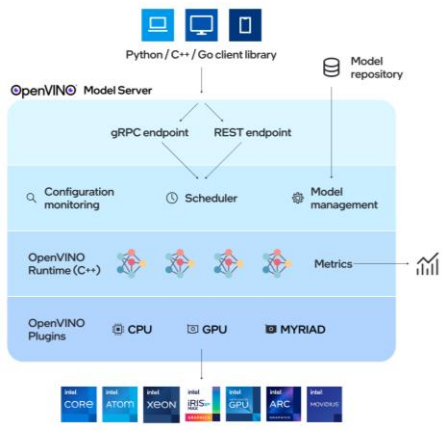
OpenVINO GenAI helps run generative models with optimized text generation, tokenization, and scheduling. The Quick-start Guide provides a compact reference for common GenAI workflows.

Inference Documentation | Compression Documentation | Reference Documentation | GenAI Quick-start Guide

OpenVINO™ Model Server (OVMS)

OpenVINO Model Server hosts models and exposes inference through standard network APIs, so applications can deploy models locally, in containers, or in Kubernetes without embedding inference logic directly.

For generative AI, OVMS provides REST and OpenAI-compatible endpoints for text generation, embeddings, reranking, and related pipeline scenarios. Recent OVMS capabilities support agentic use cases such as tool calling, multi-model workflows, and high-throughput LLM serving on Intel CPUs and GPUs.



Documentation | QuickStart Guide | Features | Demos

Join the OpenVINO Community

We welcome code contributions and feedback! Submit on GitHub and engage on GitHub discussions or our forum. Share your examples (via PR) to be featured here.